



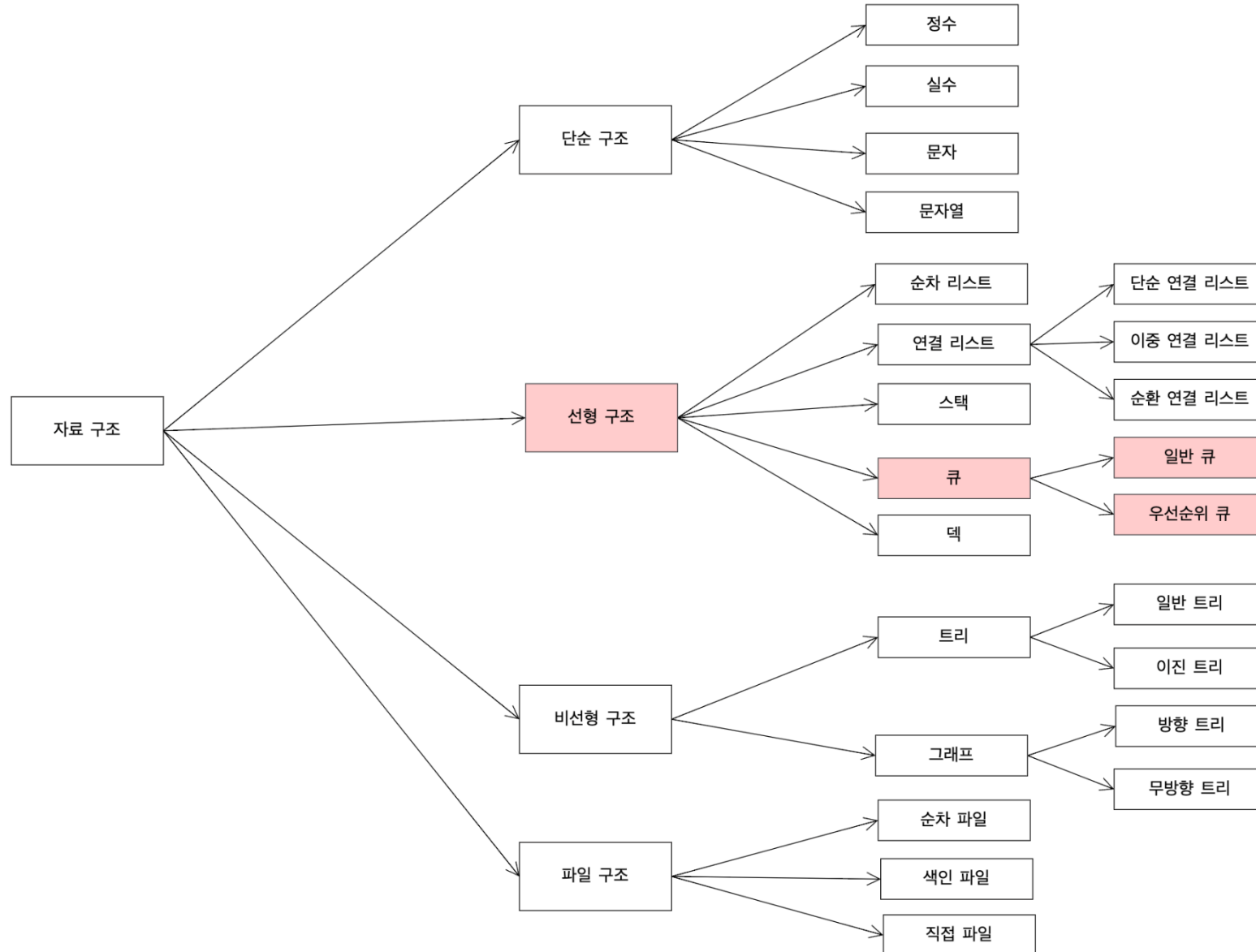
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론



Data structure



정적Static 데이터 집합

- 한번 구축되고 나면 변하지 않음

동적Dynamic 데이터 집합

- 데이터가 계속 변함
- Dictionary (Table)

삽입, 삭제, 검색을 지원하는 동적 데이터 집합을 지칭

배열, 리스트, 검색 트리, 해시 테이블, ...

(inefficient)

- 우선순위 큐Priority queue

삽입, 최우선 원소 삭제, 최우선 원소 검색을 지원하는 동적 데이터 집합

배열, 리스트, 검색 트리, 힙, ...

(inefficient)

삭제

- Table은 삭제할 원소 제공
- 우선순위 큐는 삭제할 원소 불필요 (자동 결정)
- 우선순위가 가장 높은 원소만 삭제 가능

삽입

- Table과 우선순위 큐 둘 다 삽입할 원소 제공함

원소 값 중복

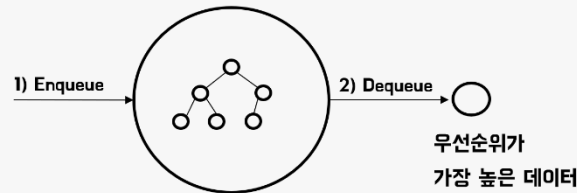
- Table은 불허
- 우선순위 큐는 허용

우선순위 큐

- 우선순위 큐는 "리스트"나 "맵"과 같이 추상적인 개념이다.
- 평범한 큐나 스택과 비슷한 축약 자료형이다.
- 각 원소들은 우선순위를 갖고 있다.
- 우선순위 큐에서, 높은 우선순위를 가진 원소는 낮은 우선순위를 가진 원소보다 먼저 처리된다.
- 리스트는 연결 리스트나 배열로 구현될 수 있는 것과 같이, 우선순위 큐는 힙이나 다양한 다른 방법을 이용해 구현될 수 있다.

우선순위 큐 (Priority Queue)

- 우선순위가 가장 큰 데이터를 반환하는 자료구조
- 추가된 데이터의 순서와 상관없이 우선순위가 가장 높은 데이터 반환



laboputer.github.io

우선순위 큐

- 큐와 구조만 동일
- 입구 출구가 구분되어 있어서 큐라고 불림
- 저장되는 데이터의 우선순위를 정하고, 그 우선순위에 따라 데이터가 저장되어 그 순서대로 데이터가 출력됨
- 들어간 순서에 상관없이 우선순위가 높은 데이터가 먼저 나오는 것
- 예를 들어

중요도가 $A > B > C$ 인 자료가 우선순위 큐에 C, A, B 순으로 저장될 경우?

저장 시에 자동 정렬되어 중요도 순으로 저장이 됨

출력은 중요도 순인 A - B - C 순으로 출력됨

우선순위 큐 추상 자료형

원소를 삽입한다

최대 원소를 알려주면서 삭제한다

최대 원소를 알려준다

우선순위 큐가 비어 있는지 확인한다

우선순위 큐를 깨끗이 비운다

최우선 순위는 최대 원소 또는 최소 원소 중 한 쪽
둘은 대칭적
여기서는 최대 원소를 최우선 원소로 가정

· 객체: n개의 element형의 우선 순위를 가진 요소들의 모임

· 연산:

- create() ::= 우선순위큐를 생성한다.
- init(q) ::= 우선순위큐 q를 초기화한다.
- is_empty(q) ::= 우선순위큐 q가 비어있는지를 검사한다.
- is_full(q) ::= 우선순위큐 q가 가득 찼는가를 검사한다.
- insert(q, x) ::= 우선순위큐 q에 요소 x를 추가한다.
- delete(q) ::= 우선순위큐로부터 가장 우선순위가 높은 요소를 삭제하고 이 요소를 반환한다.
- find(q) ::= 우선순위가 가장 높은 요소를 반환한다.

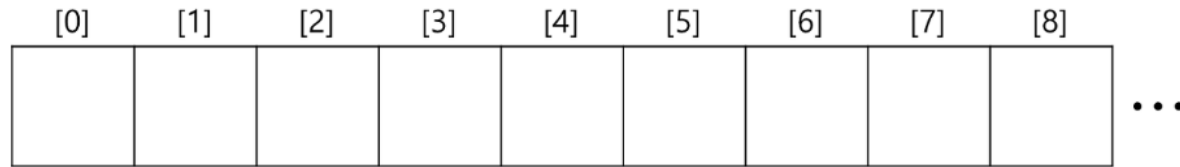
주목할 사실:

우선순위 큐에서 유일하게 접근 가능한 원소는 최대 원소 뿐

구현 방법

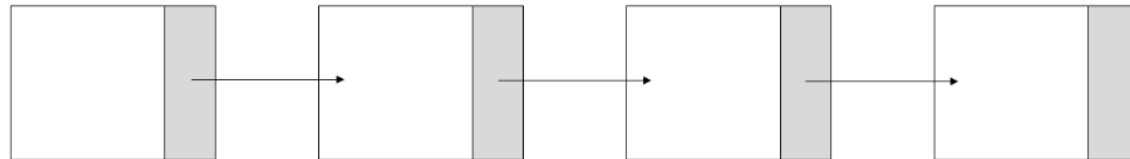
1. 배열을 이용한 우선순위 큐

- 우선순위대로 데이터들이 들어온다고 가정했을 때, 우선순위대로 아의 인덱스부터 채워나가면 되기 때문에 문제가 되지 않는다(데이터가 모두 들어오고 난 뒤에는 정렬된 상태). 하지만 연속적인 우선 순위 데이터의 입력이 보장되지 않을 경우에는 우선 순위에 맞춰 데이터를 이동시켜야 한다.



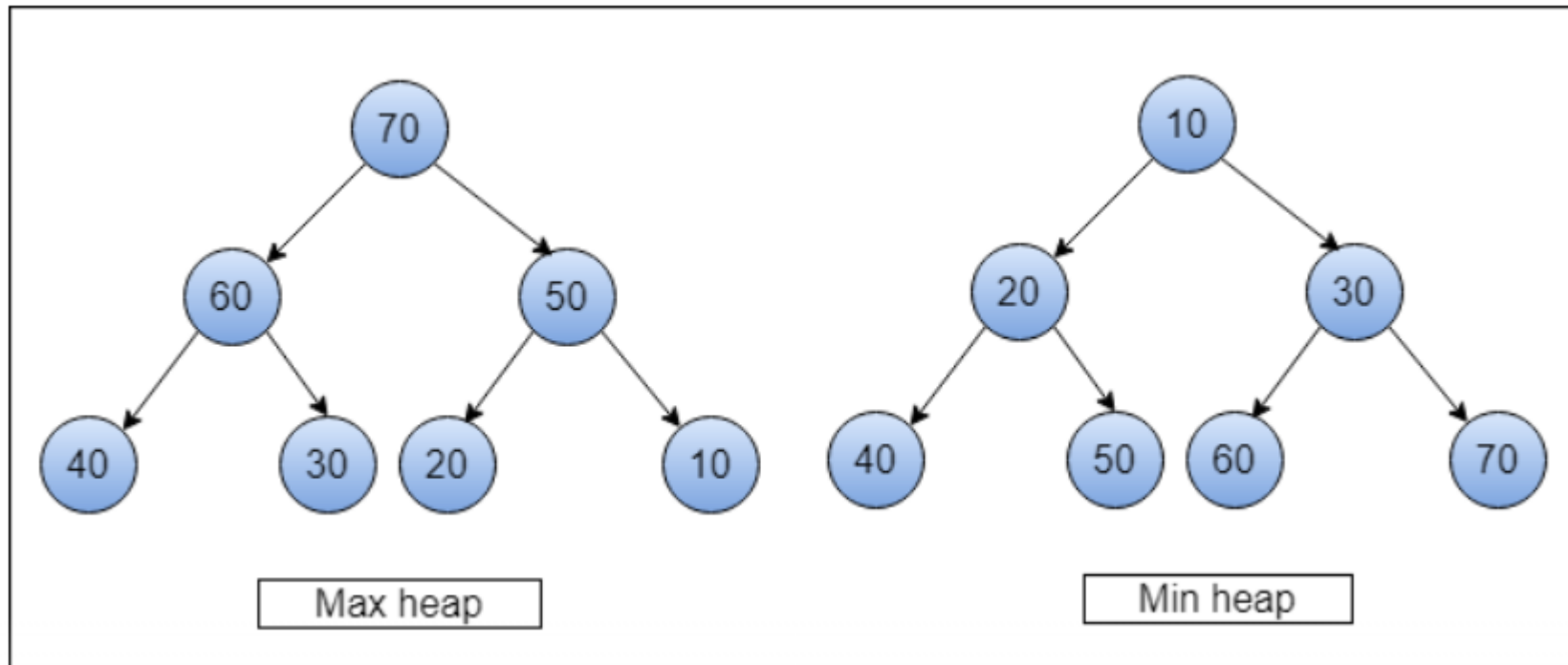
2. 연결리스트를 이용한 우선순위 큐

- 배열과 마찬가지로 데이터 삽입에 있어서 시간이 많이 걸린다. 삭제는 데이터의 이동이 필요하지 않기 때문에 배열보다 빠르다.



구현 방법

3. 힙(Heap)을 이용한 우선순위 큐
- 우선순위가 높은 데이터를 찾는 연산이 빠른 자료구조

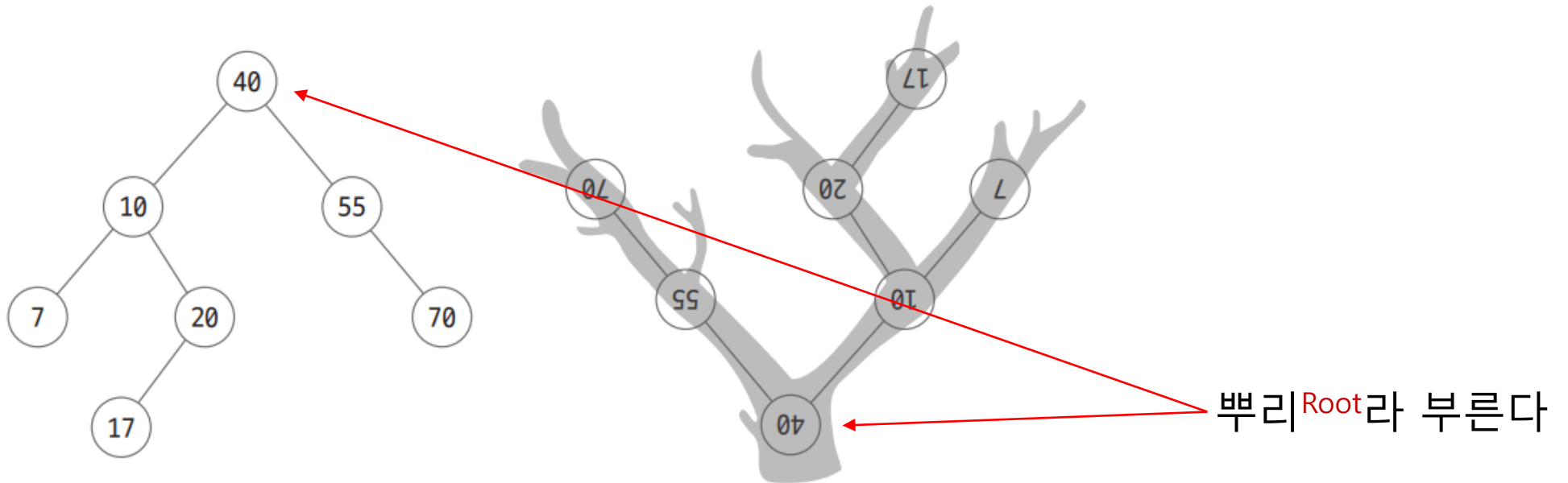


배열이나 연결리스트로 구현하지 않는 이유

- 만일 배열로 구현한다고 가정한다.
 - 우선순위가 높은 순서대로 배열의 가장 앞부분부터 넣는다면, 우선순위가 높은 데이터를 반환하는 것은 맨 앞의 인덱스를 바로 이용하면 되므로 어렵지 않다.
 - 하지만 우선순위가 중간인 것이 들어가야 하는 삽입 과정에서는 뒤의 데이터까지 인덱스를 모두 한 칸씩 뒤로 밀어야 하는 단점이 있다.
 - 최악의 경우 삽입해야 하는 위치를 찾기 위해 모든 인덱스를 탐색해야 한다.
- 만일 연결리스트로 구현한다고 가정한다.
 - 이 또한 우선 순위가 높은 순서대로 연결을 시키면, 우선순위가 높은 데이터의 반환은 배열과 마찬가지로 쉽다.
 - 하지만 연결리스트 또한 삽입의 과정 또한 배열과 마찬가지로 그 위치를 찾아야 한다. 최악의 경우 맨 끝에까지 가게 된다
- 배열/연결리스트는 구현은 쉽지만 동작이 굉장히 비효율적

이진 트리(Binary Tree)

각 노드는 0 ~ 2개까지 자식Child을 가질 수 있다



(a) 이진 트리의 예

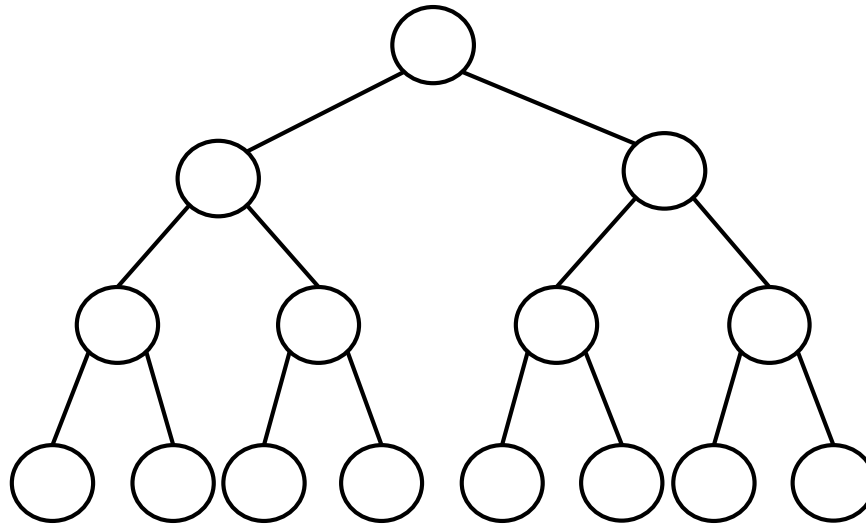
(b) (a)를 뒤집은 형태

트리의 예와 나무 비유

Binary Tree

포화 이진 트리(Full Binary Tree)

루트로부터 시작해서 모든 노드가 정확히 두 개씩의 자식 노드를 가지도록 꽉 채워진 트리

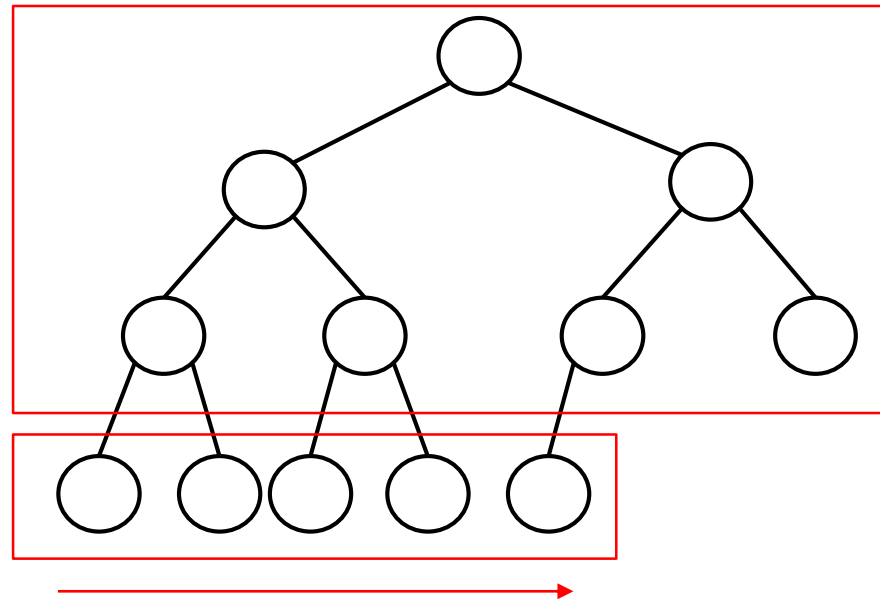


노드 수가 $2^k - 1$ 일 때만 가능
0, 1, 3, 7, 15, 31, ...

포화 이진 트리의 예

완전 이진 트리(Complete Binary Tree)

루트로부터 시작해서 가능한 지점까지 모든 노드가 정확히 두 개씩의 자식 노드를 가진다
노드의 수가 맞지 않아 full binary tree를 만들 수 없으면 맨 마지막 레벨은 왼쪽부터 채워나간다



완전 이진 트리의 예

힙(Heap)의 특징

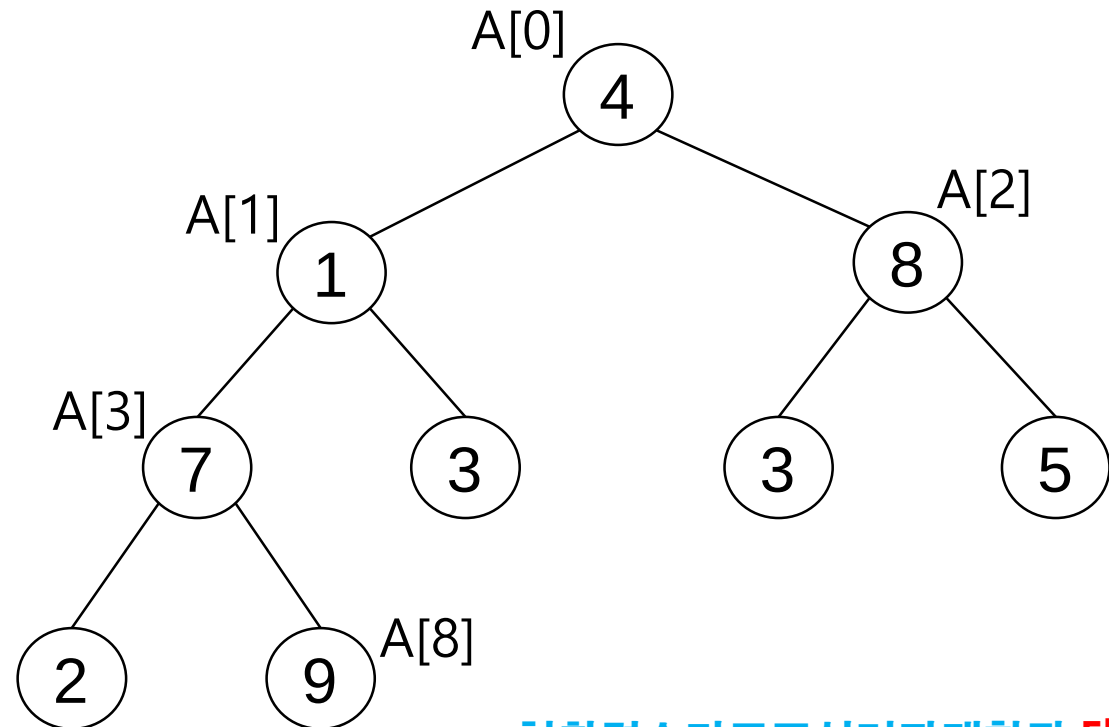
- 완전 이진 트리(Complete Binary Tree) : Heap은 완전 이진 트리의 형태를 가진다. 이는 마지막 레벨을 제외한 모든 레벨이 완전히 채워져 있고, 마지막 레벨은 왼쪽부터 채워져 있는 형태를 말한다.
- 부모-자식 노드 관계 : Heap의 부모 노드는 항상 자식 노드보다 우선순위가 높거나 같은 값을 가집니다. 이러한 특성은 최대 힙(max heap)에서는 부모 노드가 항상 자식 노드보다 큰 값을 가지며, 최소 힙(min heap)에서는 부모 노드가 항상 자식 노드보다 작은 값을 가지는 것을 의미한다.
- 힙 속성 : Heap의 모든 부모-자식 쌍에 대해 특정한 조건을 만족해야 한다. 최대 힙에서는 모든 부모 노드가 자식 노드보다 크거나 같은 값을 가져야 하며, 최소 힙에서는 모든 부모 노드가 자식 노드보다 작거나 같은 값을 가져야 한다.

힙은 배열과 안정맞춤

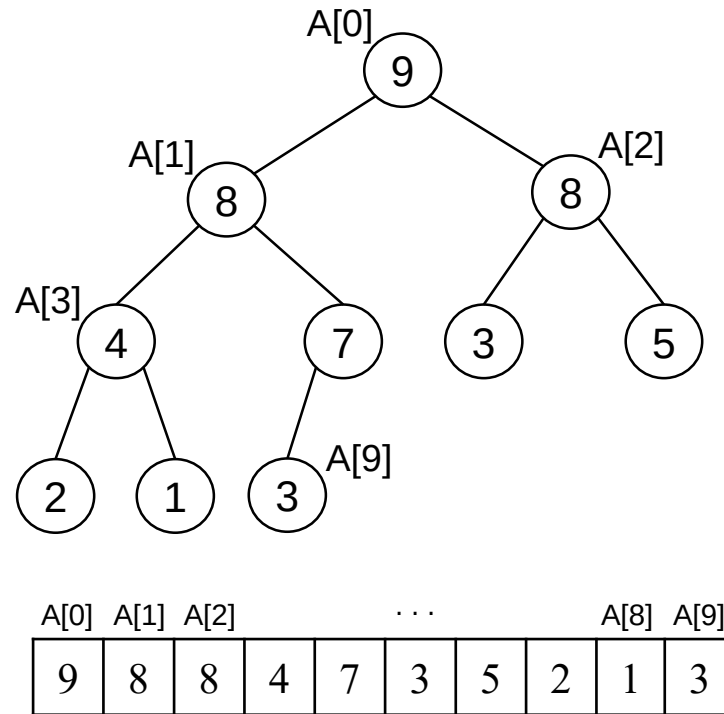
- 배열은 그 자체로 완전 이진 트리로 볼 수 있다(배열에 저장한다는 사실로 완전 이진 트리 조건은 자동 만족).
- 파이썬 내장 리스트도 배열로 간주할 수 있다.
- 단, 배열은 현재 원소가 총 몇 개인지 사용자가 관리해야 하지만 파이썬 내장 리스트는 파이썬에서 자동 관리해준다.

A[]	4	1	8	7	3	3	5	2	9
------	---	---	---	---	---	---	---	---	---

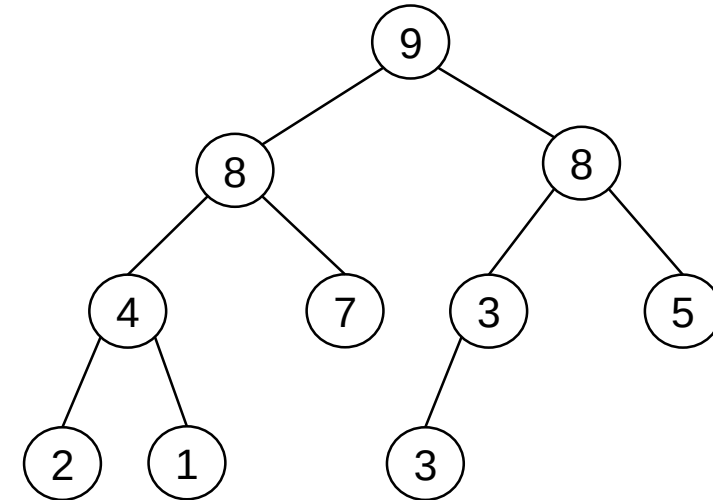
노드 i 의 자식 노드: $2i+1, 2i+2$
 노드 i 의 부모 노드: $\lfloor (i-1)/2 \rfloor$



힙의 예



10개의 원소로 구성된 힙과 대응되는 배열



힙특성은 만족하지만 완전 이진 트리를 만족하지 못하는 예

힙 조건을 만족하지 않아 힙이 아닌 예

힙 객체 구조

필드:

__A[]

작업:

insert(x)

deleteMax()

max()

buildHeap()

isEmpty()

clear()

◀ heap 원소들이 저장되는 리스트

◀ heap에 원소 x를 삽입한다

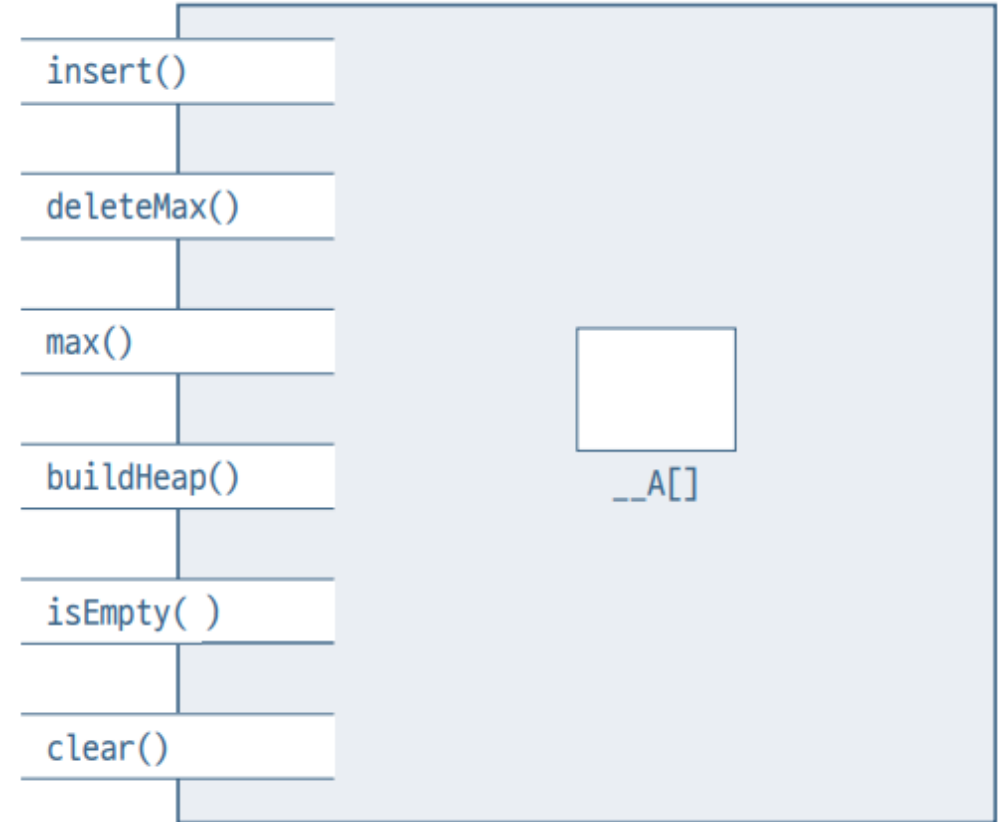
◀ heap의 최대 원소를 알려주면서 삭제한다

◀ heap의 최대 원소를 알려준다

◀ 배열 A[]를 heap으로 만든다

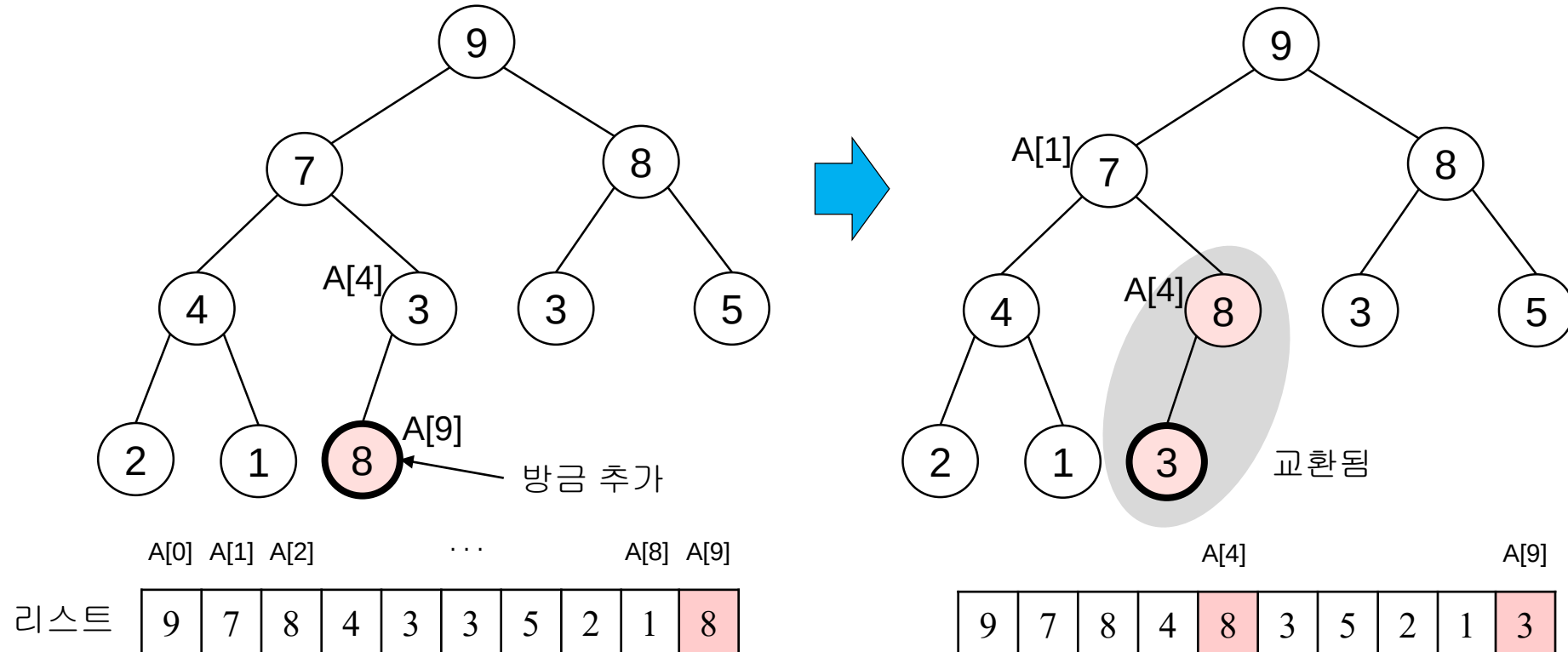
◀ heap이 빈 heap인지 알려준다

◀ heap을 깨끗이 청소한다

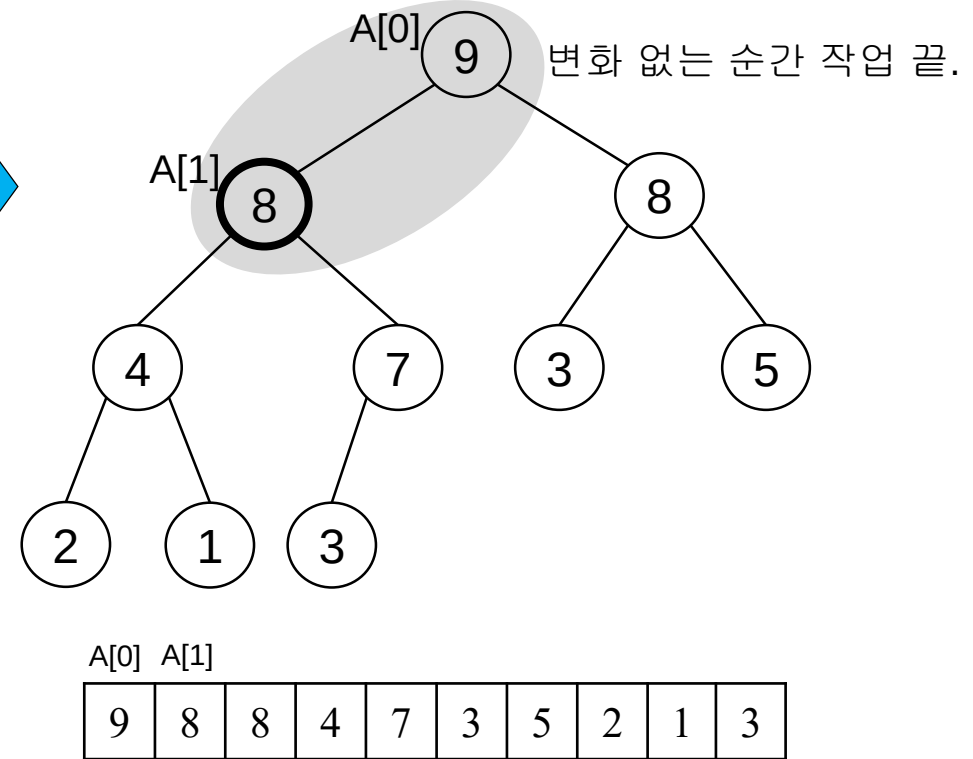
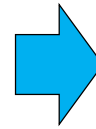
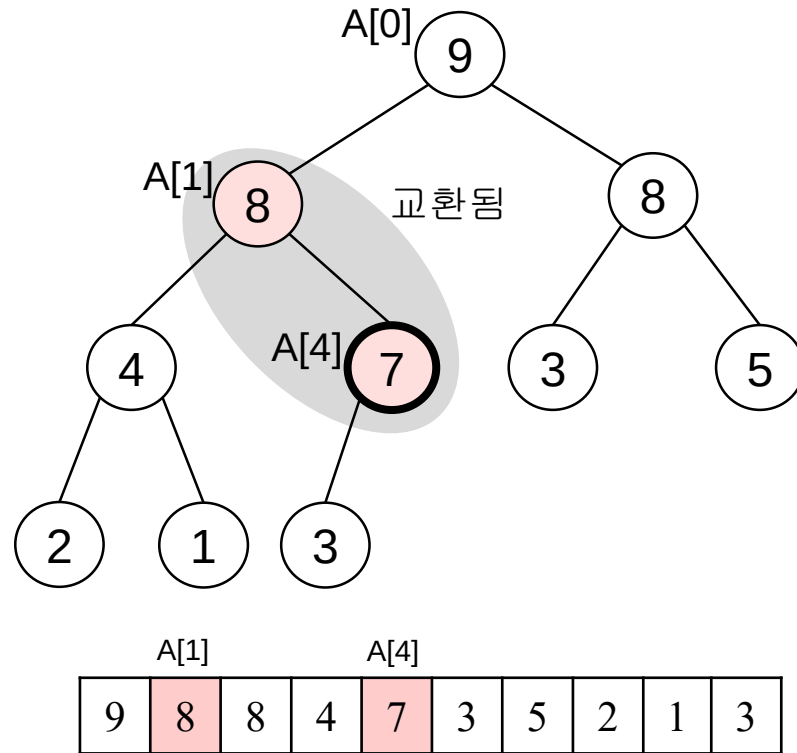


삽입Insertion

- 삽입의 예(주어진 힙에 원소 8을 삽입하는 과정)



삽입 Insertion

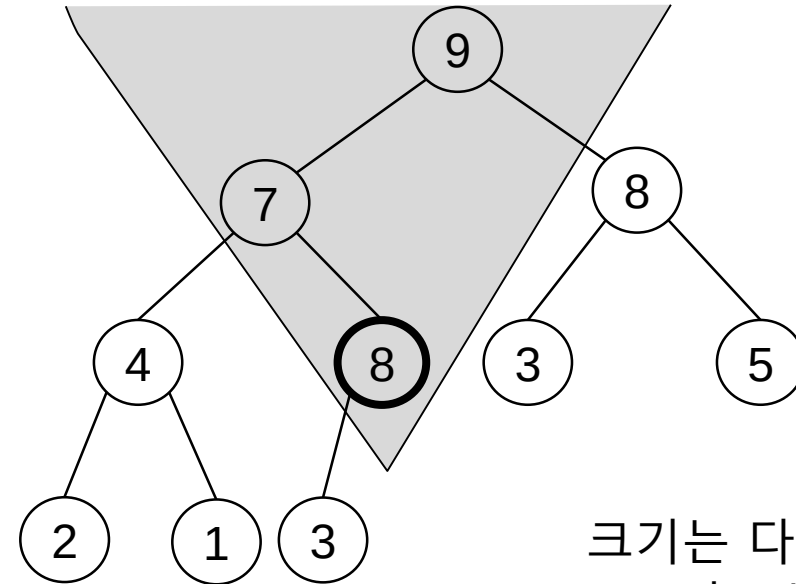
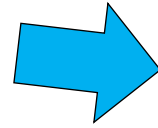
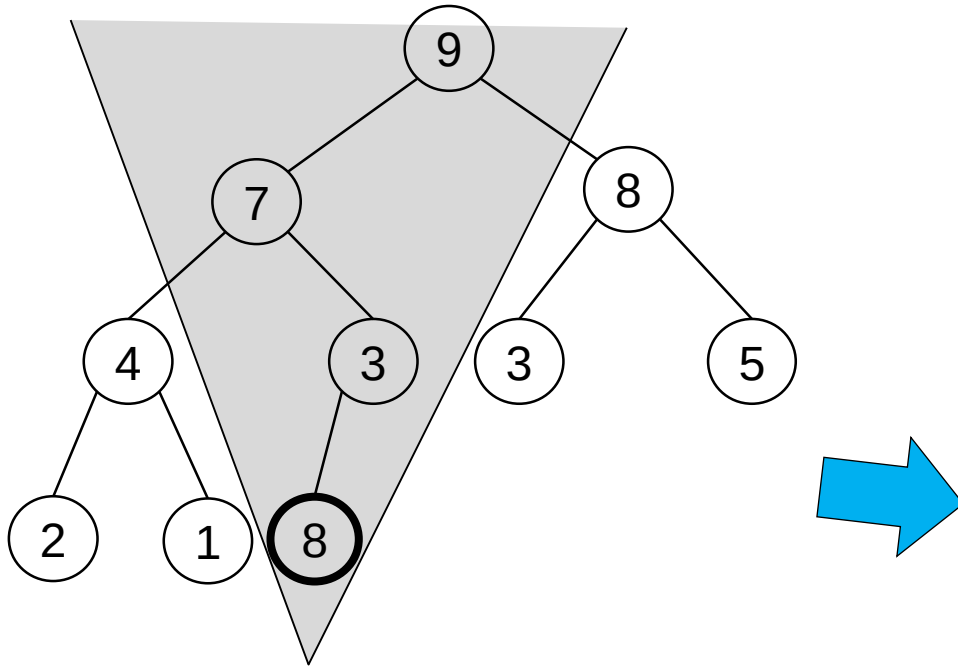


아래에서 시작하여
 조정하면서 위로 올라가는 작업을
 스며오르기 → PercolateUp이라 한다

삽입Insertion

1. 삽입 원소를 리스트의 맨 끝에 추가
2. 힙특성을 만족하도록 **스며오르기**

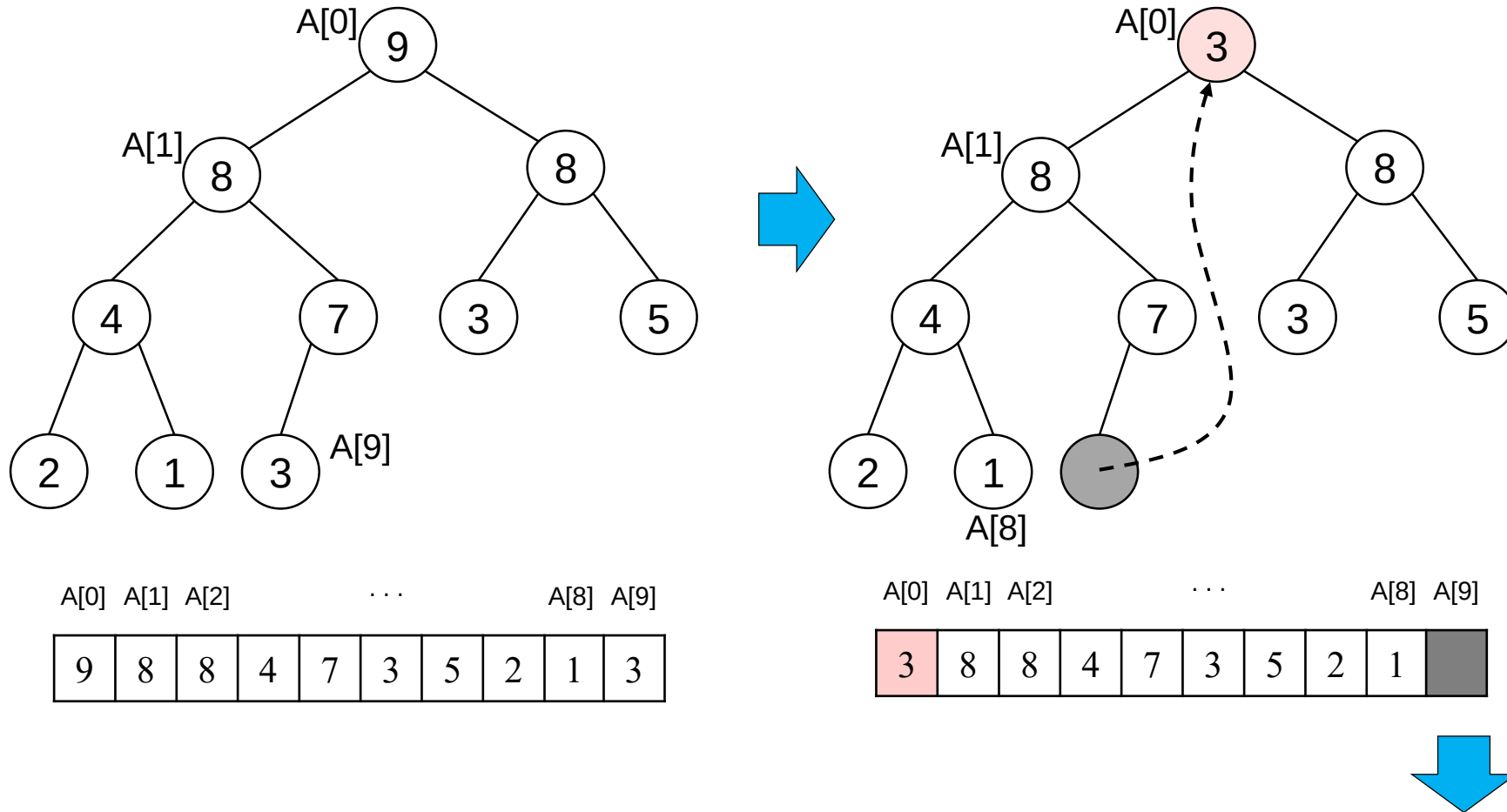
percolateUp()의 재귀적 관점



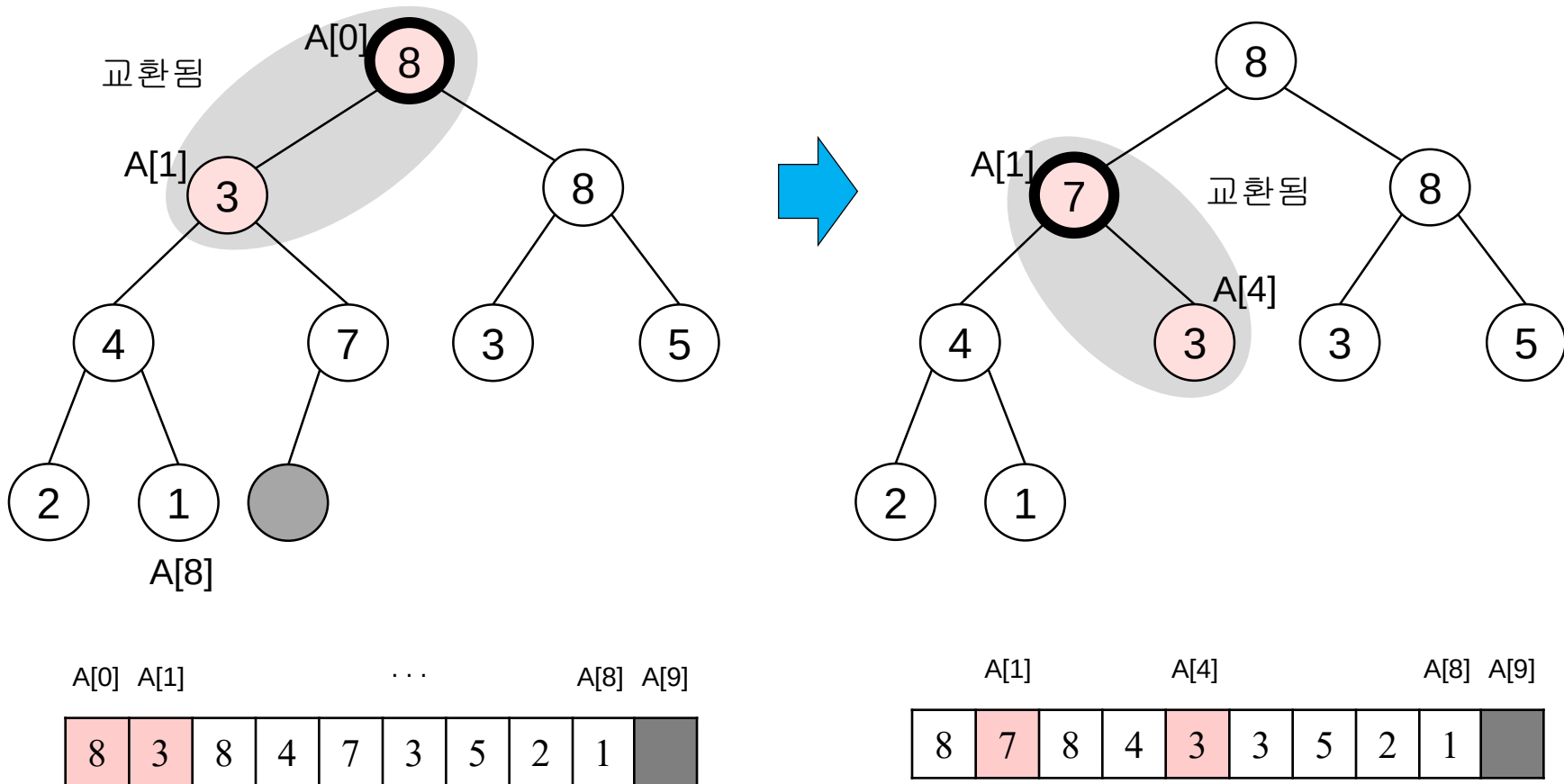
크기는 다르지만 똑같은
percolateUp 문제를 만났다

삭제 Deletion

- 삭제의 예(힙 A[0...9]에서 원소를 삭제하는 과정)

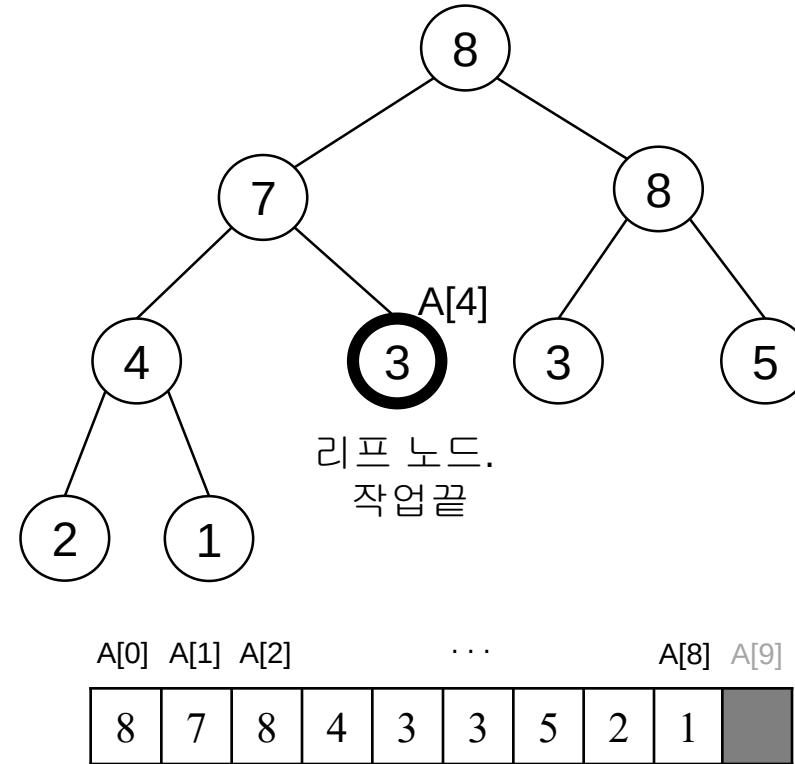


삭제 Deletion



Heap

삭제 Deletion

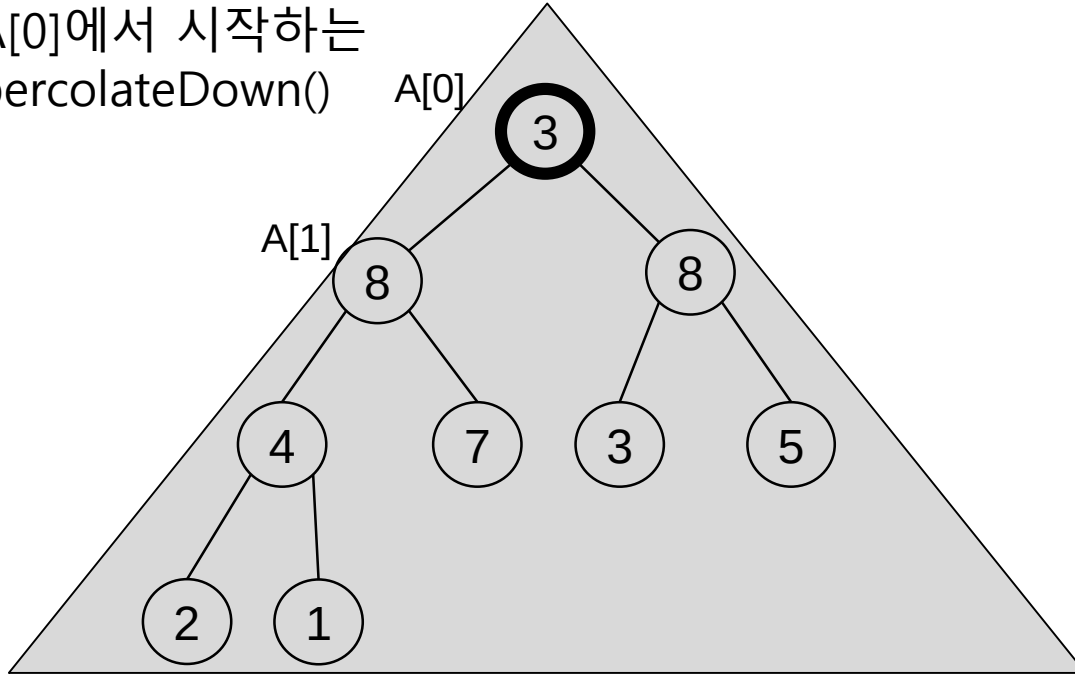


루트에서 시작하여
조정하면서 아래로 내려가는 작업을
PercolateDown이라 한다

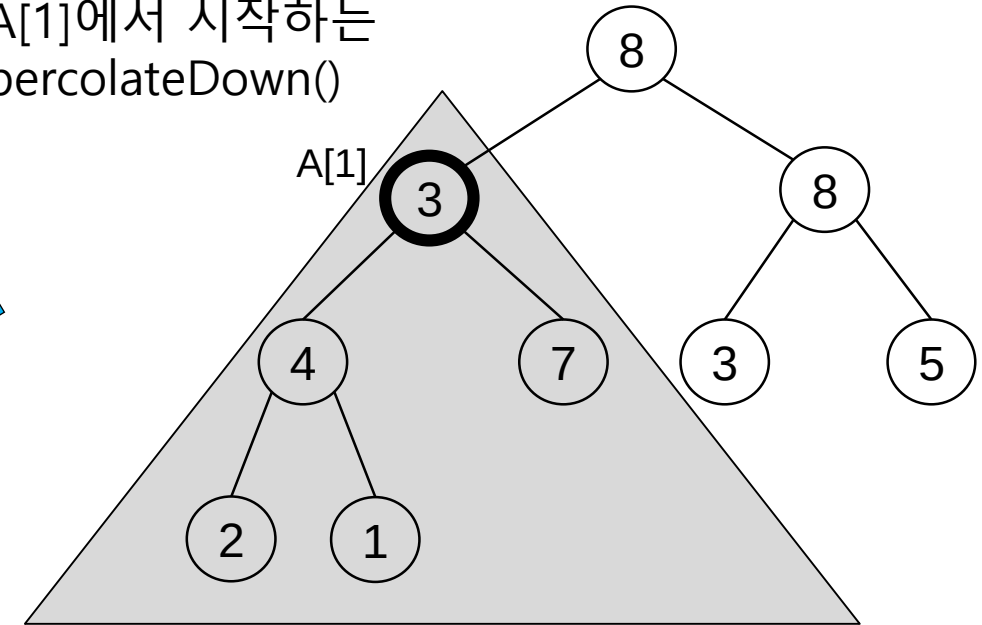

 스며내리기

percolateDown()의 재귀적 관점

A[0]에서 시작하는
percolateDown()



A[1]에서 시작하는
percolateDown()



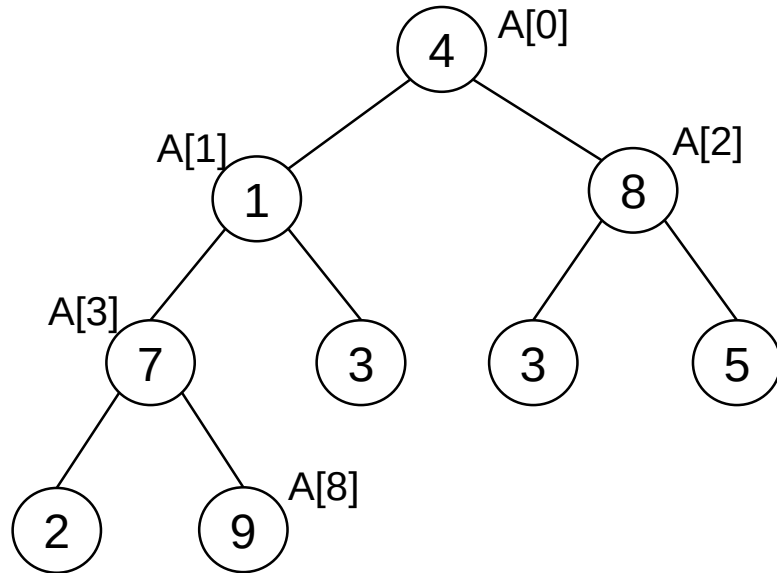
크기는 다르지만 똑같은
percolateDown 문제를 만났다

삭제 Deletion

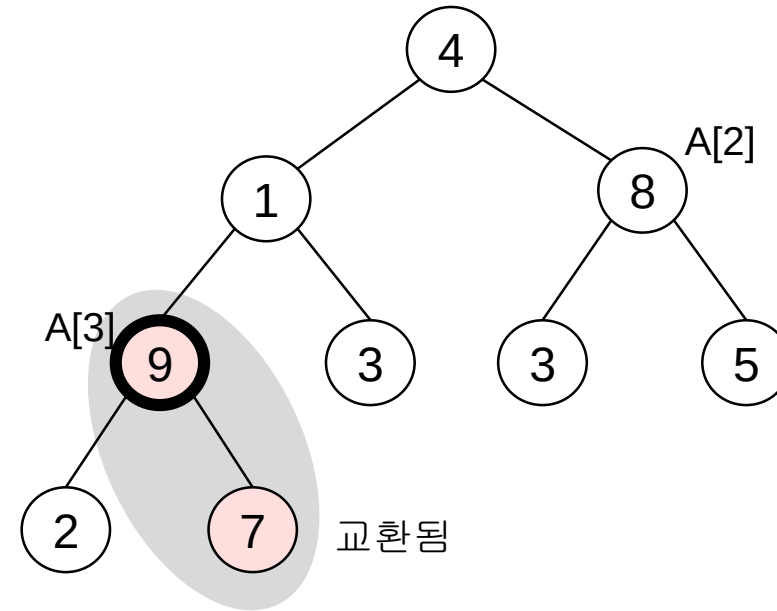
1. 루트 원소를 리턴
2. 맨 끝 원소를 루트로 이동
3. 힙 특성을 만족하도록 **스며내리기**

Heap

힙 만들기

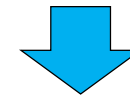


A[0]	A[1]	A[2]	...				A[8]	
4	1	8	7	3	3	5	2	9



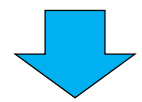
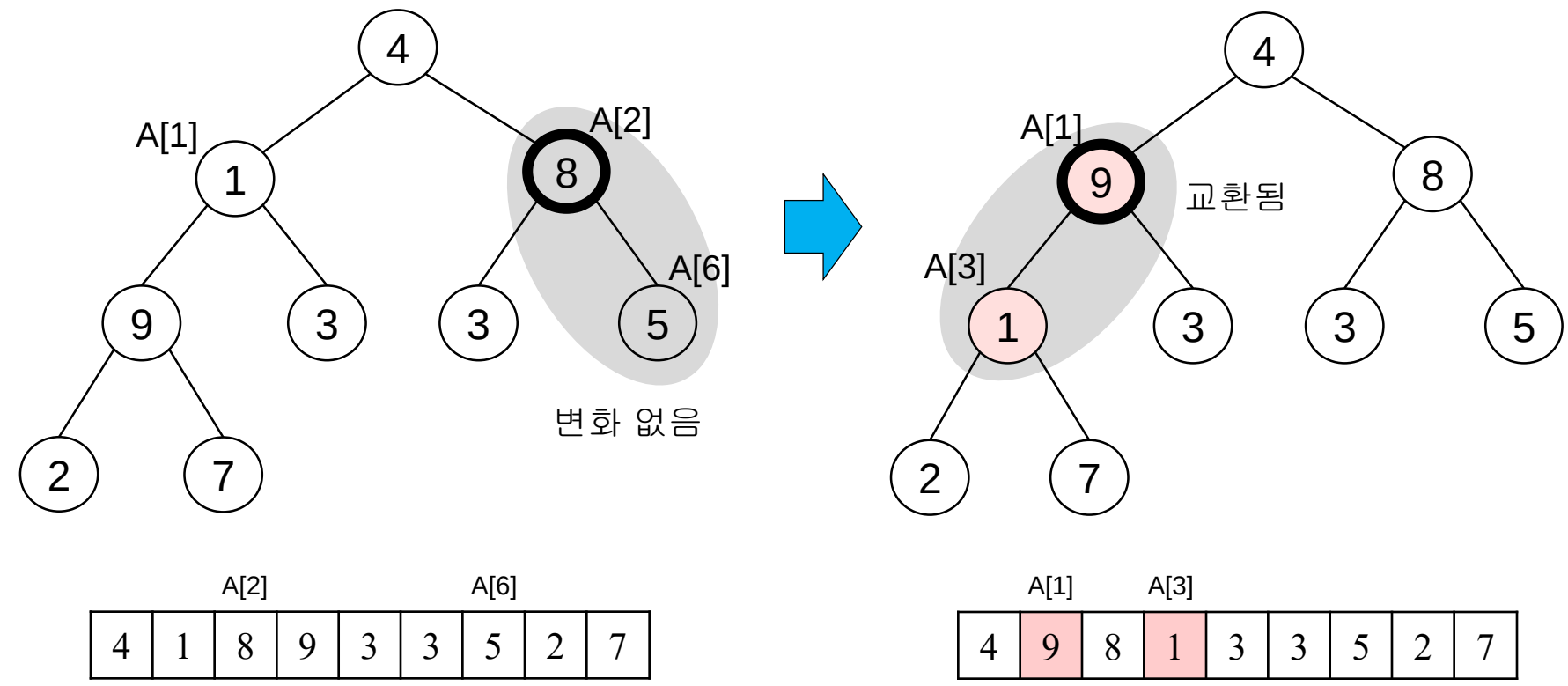
			A[3]					A[8]
4	1	8	9	3	3	5	2	7

○ : 기준 노드

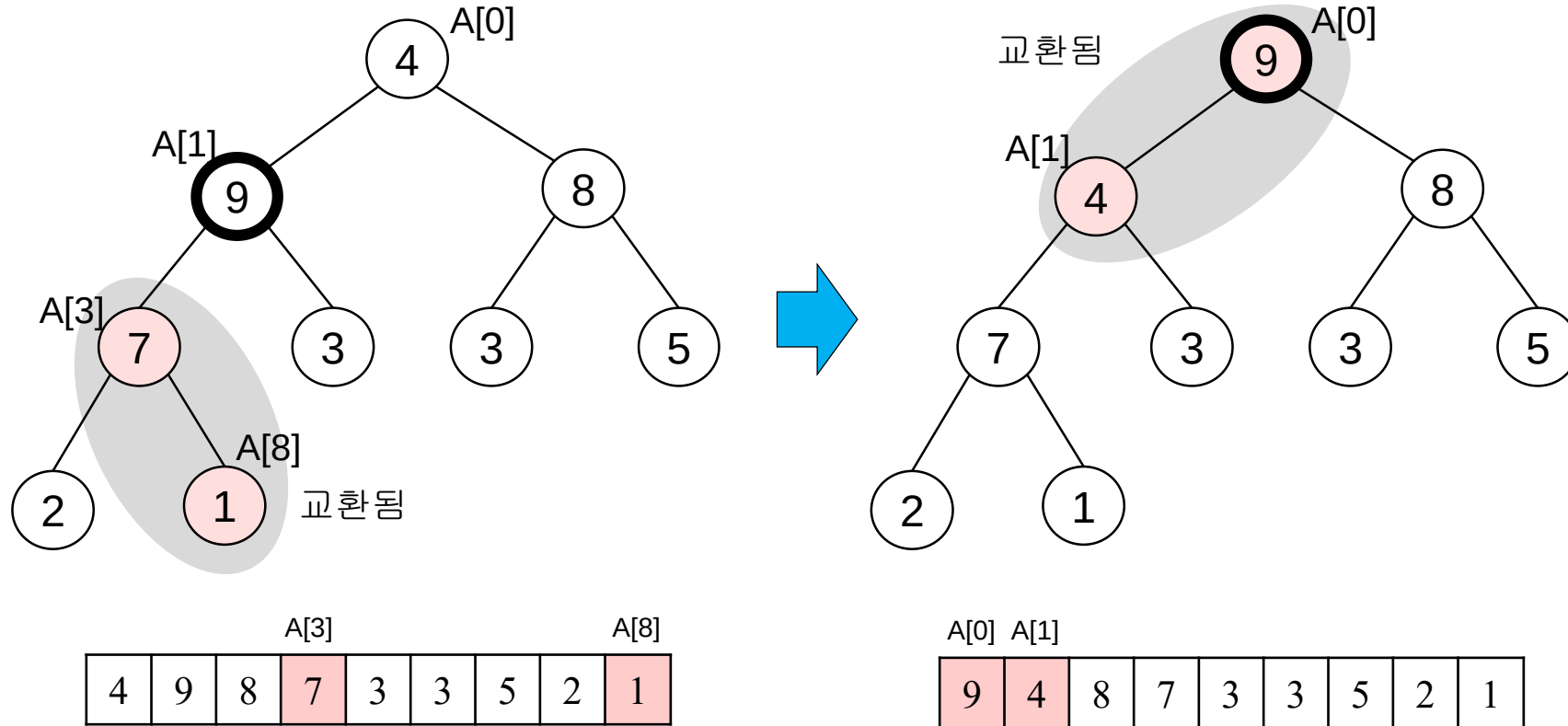


Heap

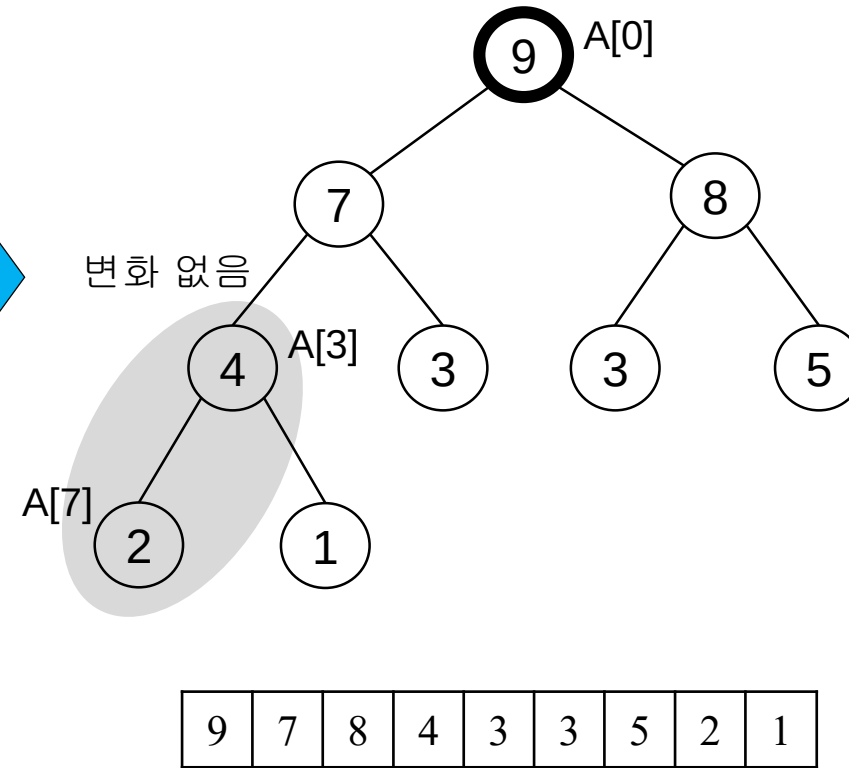
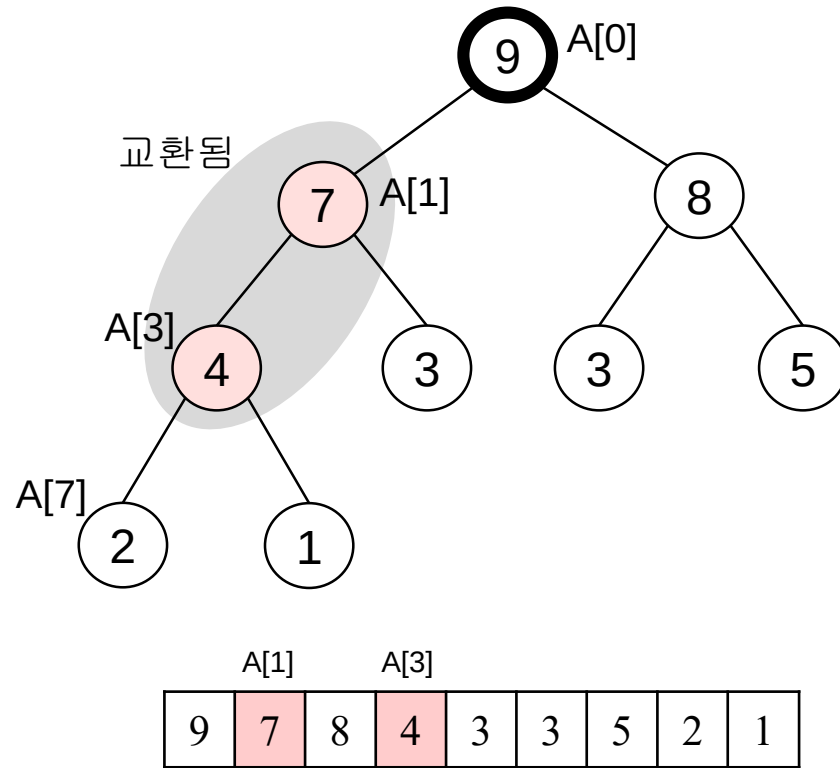
힙 만들기



힙 만들기

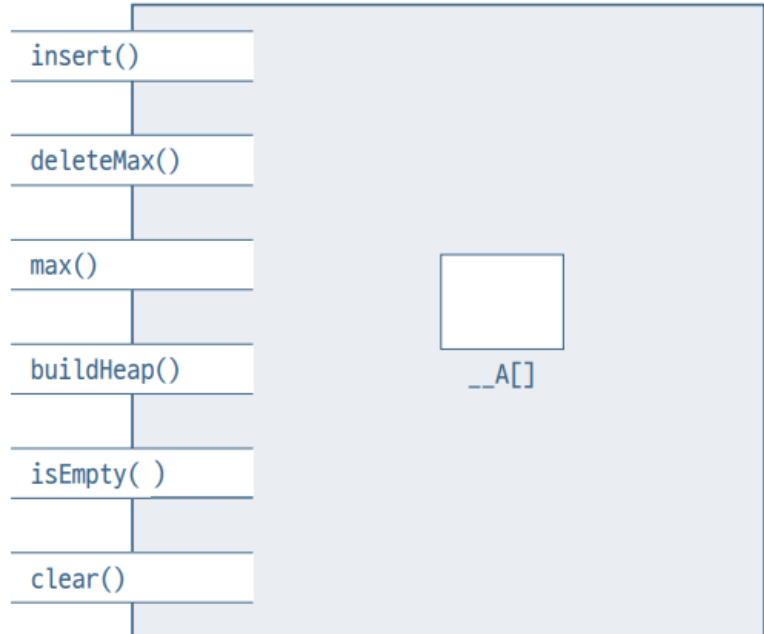


힙 만들기



Heap

파이썬 구현



힙 객체 구조

```

class Heap:
    def __init__(self, *args):
        if len(args) != 0:
            self.__A = args[0] # 파라미터로 온 리스트
        else:
            self.__A = []

    def insert(self, x):
        self.__A.append(x)
        self.__percolateUp(len(self.__A)-1)

    def __percolateUp(self, i:int):
        parent = (i - 1) // 2
        if i > 0 and self.__A[i] > self.__A[parent]:
            self.__A[i], self.__A[parent] = self.__A[parent], self.__A[i]
            self.__percolateUp(parent)

    def deleteMax(self):
        # heap is in self.__A[0...len(self.__A)-1]
        if (not self.isEmpty()):
            max = self.__A[0]
            self.__A[0] = self.__A.pop() # pop(): 리스트의 끝원소 삭제 후 리턴
            self.__percolateDown(0)
            return max
        else:
            return None
    
```

파이썬 구현

```
def __percolateDown(self, i:int):  
    # Percolate down w/ self.__A[i] as the root  
    child = 2 * i + 1 # left child  
    right = 2 * i + 2 # right child  
    if (child <= len(self.__A)-1):  
        if (right <= len(self.__A)-1 and self.__A[child] < self.__A[right]):  
            child = right # index of larger child  
        if self.__A[i] < self.__A[child]:  
            self.__A[i], self.__A[child] = self.__A[child], self.__A[i]  
  
def max(self):  
    return self.__A[0]  
  
def buildHeap(self):  
    for i in range((len(self.__A) - 2) // 2, -1, -1):  
        self.__percolateDown(i)  
  
def isEmpty(self) -> bool:  
    return len(self.__A) == 0  
  
def clear(self):  
    self.__A = []  
  
def size(self) -> int:  
    return len(self.__A)
```


파이썬 힙 자료구조

- 파이썬 heapq 모듈은 heapq (priority queue) 알고리즘을 제공한다.
- 모든 부모 노드는 그의 자식 노드보다 값이 작거나 큰 이진트리(binary tree) 구조인데, 내부적으로는 인덱스 0에서 시작해 k번째 원소가 항상 자식 원소들($2k+1$, $2k+2$) 보다 작거나 같은 최소 힙의 형태로 정렬된다.
- heapq는 내장 모듈로 별도의 설치 작업 없이 바로 사용할 수 있다.
- 힙 함수 활용하기
 - `heapq.heappush(heap, item)` : item을 heap에 추가
 - `heapq.heappop(heap)` : heap에서 가장 작은 원소를 pop & 리턴. 비어 있는 경우 IndexError가 호출됨.
 - `heapq.heapify(x)` : 리스트 x를 즉각적으로 heap으로 변환함 (in linear time, $O(N)$)

파이썬의 heapq 모듈

우선순위 큐 활용

- 다익스트라 알고리즘(Dijkstra's algorithm) : 최소 값을 찾을 때 우선순위 큐가 사용
- A* 알고리즘 : 출발지점부터 꼭짓점 까지 가는 최단 거리를 찾아줌
- 힙정렬: 많은 힙 정렬에서는 우선순위 큐를 사용
- 허프만 코딩: 문자열을 트리를 이용해 2진수로 압축하는 알고리즘이고 min-Priority를 사용
- Prim's Mininum Spanning tree
- 시뮬레이션이나 네트워크 트래픽 제어, 운영 체제의 작업 스케줄링, 수치 해석적인 계산 등에서 활용된다.

Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://laboputer.github.io/ps/2017/10/06/priorityqueue/>
- <https://docs.python.org/ko/3/library/heapq.html>
- <https://wikidocs.net/194445>